Lecture 2: From Lattices to Learning With Errors

SVP, CVP, SIS, LWE, Ring-LWE, and Module-LWE

Dr. Faisal Aslam

Lecture Notes — Session 2 (approx. 2 hours)

Purpose of this lecture. Lecture 1 built the algebraic toolkit — *Group* → *Subgroup* → *Ring* → *Field* → *Modular Arithmetic* → *Vector Space* → *Basis* → *Lattice* → *Determinant* — and ended by naming the Shortest Vector Problem (SVP) as one of the hard problems lattice cryptography rests on. This lecture finishes the job: we state SVP and CVP formally, meet the **Short Integer Solution (SIS)** and **Learning With Errors (LWE)** problems precisely, and then generalize LWE to **Ring-LWE** and **Module-LWE** — the exact hard problems underneath Kyber (ML-KEM) and Dilithium (ML-DSA). By the end, the public key of a real post-quantum scheme should look to you like "just an LWE sample," not a mysterious block of numbers.

Contents

1 Quick Recap of Lecture 1

Before building anything new, let's fix the vocabulary we'll be using constantly today.

Term	One-line meaning
Ring $(R, +, \times)$	Two operations; addition invertible, multiplication need not be
$\mathbb{Z}_q$	Integers $\{0, \ldots, q-1\}$, wraparound ("clock") arithmetic
Vector space, basis	Every point = a unique combination of basis directions
Lattice L	*Integer-only* combinations of a basis; a discrete grid in $\mathbb{R}^n$
$\det(L)$	Volume of the lattice's fundamental cell; basis-independent
$\lambda_1(L)$	Length of the shortest nonzero vector in L
Good basis vs. bad basis	Short, near-orthogonal vs. long, skewed — same lattice, different difficulty

Remark: The One Idea Everything Today Builds On

Lecture 1 ended with this sentence: *"the secret key is typically a good (short) basis, while the public key describes the same lattice using a bad basis."* Today we make that idea concrete and computational. Every hard problem in this lecture is, underneath, a question of the form: *"given a bad description of a lattice, recover something a good description would make trivial."*

2 Two Canonical Hard Problems on a Lattice

2.1 The Shortest Vector Problem (SVP)

Definition: Shortest Vector Problem (SVP)

Given a basis $B = \{b_1, \ldots, b_n\}$ for a lattice $L = \mathcal{L}(B) \subseteq \mathbb{R}^n$, find a nonzero vector $v \in L$ with $\|v\| = \lambda_1(L)$ (the minimum distance defined in Lecture 1). The **decision** version, GapSVP_γ, only asks: is $\lambda_1(L) \leq d$, or is $\lambda_1(L) > \gamma \cdot d$, for a given distance d and approximation factor $\gamma \geq 1$? (One of these two must hold; the problem promises no "messy middle" case.)

Remark: Why the "Approximation Factor" γ Matters

Finding the *exact* shortest vector ($\gamma = 1$) is extremely hard even for classical computers once n is a few hundred. But so is finding a vector even *polynomially longer* than the shortest one, as long as γ stays modest and n is large — and that weaker, "approximate" version is all cryptography actually needs to be hard. This is why you will see phrases like "SVP is hard to approximate within polynomial factors" rather than just "SVP is hard."

2.2 The Closest Vector Problem (CVP)

Definition: Closest Vector Problem (CVP)

Given a basis B for a lattice $L \subseteq \mathbb{R}^n$ and a **target point** $t \in \mathbb{R}^n$ (not necessarily in L), find the lattice point $v \in L$ minimizing $\|t - v\|$. The approximate decision version GapCVP_γ again just distinguishes "some lattice point within distance d of t" from "every lattice point is farther than $\gamma \cdot d$ from t."

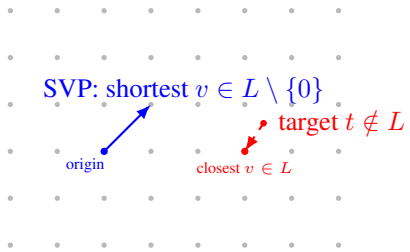


Figure 1: SVP (left, blue) asks for the shortest nonzero vector reachable inside the lattice itself. CVP (right, red) asks, for an arbitrary target point that is *not* on the grid, which grid point is nearest. Both are easy to state and easy to solve by eye in 2D — and famously hard in high dimensions with only a bad basis to work with.

Example: CVP by Eye vs. CVP by Formula

In Figure 1 you can *see* the closest grid point instantly. That's because your eye is effectively using a very good, orthogonal basis (the obvious horizontal/vertical grid directions). Algorithmically, CVP is solved by writing $t = \sum c_i b_i$ (allowing real, non-integer c_i) and **rounding each c_i to the nearest integer** — but this shortcut only gives the *true* closest point when the basis is good (short, near-orthogonal). Hand the exact same lattice to an algorithm with a bad basis instead, and naive rounding can miss the true closest point by a wide margin. This gap — correct-with-a-good-basis, unreliable-with-a-bad-basis — is precisely what CVP-based cryptography exploits.

Looking Ahead: Where SVP and CVP Are Heading

We will not attack SVP or CVP directly today. Instead, both SIS (Section 3) and LWE (Section 4) are best understood as *specific, structured* instances of SVP/CVP, dressed up so that a random instance is easy to *generate* but — conjecturally — just as hard to *solve* as the worst case. That "random instances are as hard as the worst case" guarantee is the single most important design property of modern lattice cryptography, and we return to it in Section 4.5.

3 The Short Integer Solution (SIS) Problem

Remark: Motivating SIS With a Question

Here's a question that sounds almost too easy: if I hand you a random matrix A (think: rows and columns of random numbers mod q), can you find a *short* nonzero vector x so that $Ax \equiv \vec{0} \pmod{q}$? If x could be any size at all, this would be trivial linear algebra — as long as A has more columns than rows, there are always *some* nonzero solutions (more unknowns than equations). The catch, and the entire reason this is a cryptographic problem rather than a homework exercise, is the word **short**: among the (many) solutions that exist, finding one whose entries are all small turns out to be hard once the matrix is big enough. This is SIS.

Definition: Short Integer Solution (SIS)

Fix positive integers n, $m > n$, a modulus q, and a bound β. Given a uniformly random matrix $A \in \mathbb{Z}_q^{n \times m}$, find a nonzero vector $x \in \mathbb{Z}^m$ with

$$Ax \equiv \vec{0} \pmod{q} \qquad \text{and} \qquad \|x\| \leq \beta.$$

Remark: Why This Is Secretly a Lattice Problem

Define $\Lambda^{\perp}(A) = \{\, x \in \mathbb{Z}^m : Ax \equiv \vec{0} \pmod{q} \,\}$. The name "$\perp$" is exactly what it looks like: $Ax \equiv \vec{0}$ means x pairs to zero (mod q) with *every* row of A, so $\Lambda^{\perp}(A)$ is the orthogonal complement of A's row space — the same idea as $V^{\perp}$ for ordinary vector spaces, just computed mod q instead of over $\mathbb{R}$.

A quick check confirms $\Lambda^{\perp}(A)$ is a subgroup of $\mathbb{Z}^m$ (closed under addition and negation, contains $\vec{0}$) and it is discrete (it's a subset of $\mathbb{Z}^m$) — so by Lecture 1's subgroup definition, $\Lambda^{\perp}(A)$ **is itself a lattice**. Solving SIS is exactly finding a short nonzero vector in $\Lambda^{\perp}(A)$ — i.e., **SIS is SVP**, restricted to this specific, algebraically-generated family of lattices. The reason cryptographers like this family is that generating a *random* instance is as easy as generating a random matrix A, yet the resulting lattice is (conjecturally) exactly as hard to solve SVP on as the worst-case lattices Lecture 1 discussed abstractly.

Example: A Tiny SIS Instance by Hand

Let $n = 1$, $m = 4$, $q = 17$, and $A = \begin{pmatrix} 3 & 5 & 8 & 1 \end{pmatrix} \in \mathbb{Z}_{17}^{1\times 4}$. We want a short, nonzero, *integer* vector $x = (x_1, x_2, x_3, x_4)$ with $3x_1+5x_2+8x_3+x_4 \equiv 0 \pmod{17}$. Try $x = (1, 1, -1, -16) \to$ too long. Instead try $x = (1, -1, 0, 2)$: $3(1) + 5(-1) + 8(0) + 1(2) = 3 - 5 + 0 + 2 = 0 \equiv 0 \pmod{17}$. Success, and $\|x\|$ is small (each entry is ≤ 1 in absolute value except the last, which is 2). This x is a valid, short solution — with $m = 4$ columns squeezed against only $n = 1$ modular equation, short solutions are guaranteed to exist by a pigeonhole argument (more columns than rows means lots of redundancy); the point of SIS being *hard* is that finding one is nontrivial once n, m are cryptographically large (hundreds to thousands), not that one fails to exist.

Remark: What SIS Is Used For

SIS is the workhorse behind lattice-based **hash functions** and was historically the basis of early lattice signature schemes. Section 3.1 below builds the hash-function connection explicitly, right now, while the SIS instance above is still fresh. You will meet SIS's signature-adjacent side again in Session 3 (Dilithium/ML-DSA) and again in Session 5, since Dilithium's fault-attack surface is closely tied to how it uses a SIS-flavored rejection-sampling step.

3.1 A Concrete Application: SIS-Based Hash Functions

Remark: What a Hash Function Needs

A cryptographic hash function takes an input of some size and **compresses** it down to a fixed, shorter output, with one crucial security property: **collision resistance** — it should be hard to find two *different* inputs $x_1 \neq x_2$ that hash to the *same* output. Everything below is aimed at exactly one goal: building a compressing function whose collision resistance follows directly, and provably, from SIS being hard — no separate, from-scratch security assumption required.

Definition: The Ajtai-Style SIS Hash Function

Fix a uniformly random matrix $A \in \mathbb{Z}_q^{n \times m}$ (exactly the SIS matrix above), with $m > n \log_2 q$ so the map below is genuinely compressing. Define $h_A : \{0,1\}^m \to \mathbb{Z}_q^n$ by

$$h_A(x) = Ax \bmod q.$$

The input x is an m-bit string (so there are 2^m possible inputs); the output lives in $\mathbb{Z}_q^n$ (only q^n possible outputs). Since $m > n \log_2 q$ forces $2^m > q^n$, the function is squeezing a strictly bigger set down into a strictly smaller one — exactly the compression a hash function needs, and exactly why some collision is guaranteed to exist by the pigeonhole principle alone (finding one is the hard part).

Remark: Why Collision Resistance *Is* SIS, Not Just *Like* SIS

Suppose an attacker finds a collision: two distinct inputs $x_1 \neq x_2 \in \{0,1\}^m$ with $h_A(x_1) = h_A(x_2)$, i.e. $Ax_1 \equiv Ax_2 \pmod q$. By linearity, $A(x_1 - x_2) \equiv \vec{0} \pmod q$. Set $x = x_1 - x_2$. Then:

- $x \neq \vec{0}$, since $x_1 \neq x_2$.
- $Ax \equiv \vec{0} \pmod q$ — exactly the SIS equation.
- x is automatically **short**: each coordinate of $x_1, x_2 \in \{0,1\}^m$ is 0 or 1, so each coordinate of $x = x_1 - x_2$ is in $\{-1, 0, 1\}$, giving $\|x\| \leq \sqrt{m}$ — comfortably within any reasonable SIS bound β.

So **a collision in h_A is, verbatim, a solution to the SIS instance defined by the same matrix A.** Turning this around: if SIS is hard for A, then finding a collision in h_A is exactly as hard — there is no weaker link elsewhere in the construction. This is the entire appeal of building a hash function this way: security reduces to a single, already-studied lattice problem, with nothing extra to trust.

Example: A Collision, Found and Verified by Hand

Reuse $n = 1$, $m = 4$, $q = 17$, $A = \begin{pmatrix} 3 & 5 & 8 & 1 \end{pmatrix}$ from the SIS worked example above, and the short SIS solution already found there, $x = (1, -1, 0, 2)$. That x has entries outside $\{-1, 0, 1\}$ in one coordinate (the 2), so it isn't itself a difference of two bit-strings — but it illustrates the mechanism cleanly, and a genuine bit-string collision is built the same way. Take

$$x_1 = (1, 0, 0, 1), \qquad x_2 = (0, 1, 0, 0) \qquad \text{(both in } \{0,1\}^4\text{)}.$$

Compute both hashes mod 17:

$$h_A(x_1) = 3(1) + 5(0) + 8(0) + 1(1) = 4, \qquad h_A(x_2) = 3(0) + 5(1) + 8(0) + 1(0) = 5.$$

These don't collide ($4 \neq 5$) — so x_1, x_2 is *not* a collision, only a candidate pair, exactly as most randomly-chosen pairs will not collide. This is the expected, normal outcome: with $q = 17$ this toy instance barely compresses at all ($m = 4$ bits down to one symbol in $\{0, \ldots, 16\}$), so collisions are not hard to stumble on by search, but the *mechanism* — take any actual collision, subtract, get a short nonzero SIS solution — is exactly what the remark above proves in general, regardless of how easy or hard this particular toy-sized search happens to be.

Remark: Why This Only Works Because of Compression

It's worth being precise about where $m > n \log_2 q$ was used. Without that condition, h_A might be injective (no collisions could exist at all, even in principle), and the whole security question would be vacuous. With it, collisions are *guaranteed to exist* (pigeonhole, as noted in the def-box) — the hardness claim is never "no collision exists," only "no one can *find* one efficiently." This mirrors ordinary cryptographic hash functions like SHA-256 exactly: collisions must exist (infinite inputs, finite outputs), and security only ever means "hard to find," never "impossible."

Fact: $h_A(x) = Ax$ vs. SHA-2/SHA-3: Two Different Kinds of Confidence

- **SHA-2/SHA-3:** security is *empirical* — decades of public cryptanalysis have failed to find a collision. Fast, compact, no known quantum weakness worth designing around (Grover only gives a quadratic speedup). The default choice for hashing in practice.
- **h_A:** security is a *theorem* — a collision provably yields a solution to worst-case SIS/lattice problems, not just this one instance. The cost is size and speed: a large public matrix A and a matrix–vector multiply per hash, versus a few dozen fast bitwise operations.
- **Why bother, then:** h_A is *linear* in x, so it composes algebraically with other lattice constructions — homomorphic operations, commitments, zero-knowledge proofs — in ways SHA-2/SHA-3's deliberately nonlinear internals cannot. Its role is as a building block *inside* lattice-based protocols, not as a drop-in replacement for general-purpose hashing.

Exercise: Practice

Using the same $A = (3, 5, 8, 1) \in \mathbb{Z}_{17}^{1\times 4}$: (a) compute $h_A(x)$ for $x_1 = (1, 1, 0, 0)$ and $x_2 = (0, 0, 1, 1)$; do they collide? (b) Find, by inspection or trial, a genuine colliding pair $x_1 \neq x_2 \in \{0, 1\}^4$ with $h_A(x_1) = h_A(x_2)$. (c) For your pair from part (b), compute $x = x_1 - x_2$ and confirm directly that $Ax \equiv 0 \pmod{17}$ — i.e., confirm your collision really does hand you a SIS solution, exactly as the argument above claims it must.

Remark: SIS vs. LWE: Why We Need Both

SIS and LWE are built from the same raw material — a random matrix A — but they are hard for different reasons and do different jobs. SIS's hardness is about *collisions*: it's hard to find a short x with $Ax \equiv \vec{0}$, which is exactly the guarantee **hash functions** (Section 3.1 above) and (lattice) **signatures** need — there's no secret to hide, only "no two short inputs collide." LWE's hardness is about *concealment*: without noise, $b = As$ is solved instantly by ordinary linear algebra (Section 4.1's box below); the noise e is precisely what breaks that shortcut, and it creates an asymmetry — whoever knows s can cancel/tolerate the noise, while anyone else just sees (A, b) that looks uniformly random. That asymmetry is what **encryption** needs, and SIS alone can't give it. The two problems are even dual views of the same A: SIS lives on the kernel lattice $\Lambda^{\perp}(A)$ (Section 3 above), LWE lives on the image lattice $\Lambda_q(A)$ (Section 4.5 below) — one random matrix, two lattices, two different cryptographic jobs.

4 Learning With Errors (LWE)

4.1 The Idea, in Plain Language

Imagine someone hands you a stack of linear equations in an unknown secret vector s — but each equation's right-hand side has been nudged by a small, random amount of noise before you see it. Solving *noise-free* linear equations is easy (that's just linear algebra — Gaussian elimination). Solving them once every equation is slightly wrong, and you don't know by how much, turns out to be extremely

hard. That is the entire idea of LWE.

Remark: Why Not Just Solve It With Linear Algebra?

This is the single most common point of confusion about LWE, so it's worth working through a tiny concrete case. Suppose $n = 2$ and, with no noise at all, we're given:

$$1 \cdot s_1 + 2 \cdot s_2 = 8, \qquad 3 \cdot s_1 + 1 \cdot s_2 = 9.$$

Two equations, two unknowns — solve normally (e.g. elimination) and you get $s_1 = 2,\ s_2 = 3$ exactly, every time, no matter how large n gets, as long as you have n independent equations. **Linear algebra doesn't care how big the system is** — Gaussian elimination scales smoothly to thousands of unknowns. Now add noise: suppose the equations you're actually given are

$$1 \cdot s_1 + 2 \cdot s_2 = 8 + e_1, \qquad 3 \cdot s_1 + 1 \cdot s_2 = 9 + e_2,$$

for some small, unknown $e_1, e_2 \in \{-1, 0, 1\}$ that you are never told. Now elimination doesn't work: any attempt to "solve exactly" is solving for the *wrong* thing, since the right-hand sides are off by an unknown amount. You could try every combination of e_1, e_2 and re-solve each time — for two small error values that's manageable, but for hundreds of equations, each with its own small unknown error, the number of combinations to check explodes exponentially. That explosion, not any deep mystery, is where LWE's hardness lives: **noise breaks the one tool (linear algebra) that would otherwise make this trivial**, and no comparably efficient tool has been found to replace it.

4.2 Formal Definition

Definition: Learning With Errors (LWE)

Fix a secret dimension n, a number of samples m (typically $m \gg n$ in practice), a modulus q, and an error distribution χ over $\mathbb{Z}_q$ (Section 4.3 pins down the usual choice of χ). Exactly as with SIS (Section 3), the problem starts from a **uniformly random matrix** — fix $A \in \mathbb{Z}_q^{m \times n}$ chosen uniformly at random. Let $s \in \mathbb{Z}_q^n$ be a fixed, unknown **secret vector**, and let $e \in \mathbb{Z}_q^m$ be an **error vector** whose m entries are drawn independently from χ. Define

$$b \;=\; As + e \pmod{q} \;\in \mathbb{Z}_q^m.$$

- **Search-LWE:** given (A, b), find s.
- **Decision-LWE:** given (A, b), decide whether b was built this way, or b is instead a *uniformly random* vector in $\mathbb{Z}_q^m$, independent of A.

Row by row, this is exactly the "many samples" description. Write $a_i \in \mathbb{Z}_q^n$ for the i-th row of A, and b_i for the i-th entry of b. Then $b_i = \langle a_i, s \rangle + e_i \pmod{q}$ is one LWE sample, for $i = 1, \ldots, m$ — and A is nothing more than these m row-vectors $a_1, \ldots, a_m$ stacked on top of each other. "Many noisy inner-product samples" and "one noisy matrix-vector equation" are the *same object*, just described two ways; every worked example below moves freely between the two descriptions.

Remark: Yes — LWE Has a Matrix Too, Just Playing the Opposite Role From SIS

It's easy to walk away from Section 3 thinking "SIS has a matrix, LWE doesn't" — especially since LWE is usually introduced sample-by-sample first. That impression is wrong: **both problems start from the exact same raw ingredient, a uniformly random matrix over $\mathbb{Z}_q$.** What differs is the *shape* of the matrix and what you're asked to do with it:

- **SIS:** $A \in \mathbb{Z}_q^{n \times m}$ is *wide* (more columns than rows, $m > n$). You search for a short nonzero x in A's *kernel*: $Ax \equiv \vec{0}$. Nothing is hidden — the matrix is entirely public, and the whole difficulty is finding one particular short vector among the many that satisfy the equation.
- **LWE:** $A \in \mathbb{Z}_q^{m \times n}$ is *tall* (at least as many rows as columns, $m \geq n$, usually $m \gg n$). You're handed a point b that sits *near* A's *image* (near some As), nudged off by small noise e, and asked to recover which s produced it, or even just to tell that b isn't uniform.

Same raw material, opposite structural question: SIS asks "what short vector does A kill?"; LWE asks "what secret does A almost reveal?" This is also exactly why Lecture 1's subgroup/lattice machinery applies to *both*: SIS's matrix generates the kernel lattice $\Lambda^{\perp}(A)$ (Section 3), while LWE's matrix generates an image lattice $\Lambda_q(A)$ (Section 4.5 below) — two different lattices built from the same kind of object.

A notational warning. Some sources — including Chris Peikert's *Lattices in Cryptography* lecture notes (Lecture 13, "Learning With Errors"), which Lecture 1's bibliography already points to — state LWE with the *transposed* convention: a *wide* matrix $A \in \mathbb{Z}_q^{n \times m}$ (columns, not rows, are the samples), a secret *row* vector s, and $b^T = s^T A + e^T$. This is mathematically identical to the defbox above — just transpose everything, and "columns of A are samples" becomes "rows of A are samples." We fix the row-convention above throughout this course because it matches how Lecture 3 writes Kyber's $\vec{t} = A\vec{s} + \vec{e}$ and Lecture 4's ML-DSA $\vec{t} = A\vec{s}_1 + \vec{s}_2$; just be aware that flipping between sources' conventions is normal in this literature, and always check which axis a given source calls "n" and which it calls "m" before comparing formulas.

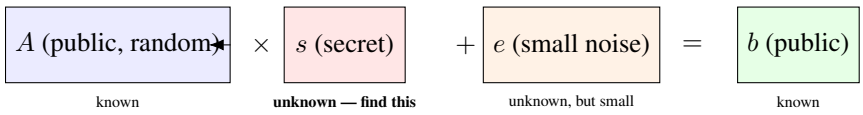


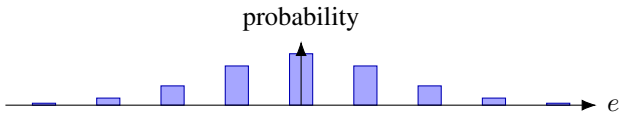
Figure 2: The anatomy of one LWE equation $b = As + e \pmod{q}$. An attacker sees A and b (both public/known) and must recover s, without ever being told e — only that e is small. Without the noise term e, this would be ordinary linear algebra and trivial to invert; the noise is precisely what makes it hard.

4.3 What the Error Distribution χ Actually Is

We've been saying "a small error" without pinning down what "small" means or where e_i actually comes from. Time to fix that.

Definition: Discrete Gaussian Distribution (the usual choice of χ)

For a parameter $\sigma > 0$ (roughly, how spread out the noise is), the discrete Gaussian distribution over $\mathbb{Z}$ assigns to each integer x a probability proportional to $\exp(-x^2/2\sigma^2)$ — the same bell-curve shape as the ordinary (continuous) Gaussian, just evaluated only at integers and renormalized so the probabilities sum to 1. In practice this means: $e = 0$ is the single most likely outcome, $e = \pm 1$ is somewhat less likely, $e = \pm 2$ less likely still, and values far from 0 (say $|e| > 6\sigma$) are so improbable they essentially never occur. χ is this distribution, and each e_i in an LWE sample is drawn independently from it.



Remark: Why a Bell Curve, and Not (Say) a Coin Flip $\{-1, 0, 1\}$?

Our worked examples below use a crude stand-in ($e \in \{-1, 0, 1\}$, each roughly equally likely) purely because it's easy to compute by hand. Real schemes use something bell-curve-shaped (a true discrete Gaussian, or in Kyber's case a closely related and cheaper-to-sample *centered binomial distribution*) for a precise mathematical reason: the hardness proofs connecting LWE back to worst-case lattice problems (Regev's reduction, Section 4.5) require the noise to behave like a Gaussian. Swap in a different-shaped distribution and those proofs may simply no longer apply — "small errors" alone isn't enough; their *shape* matters too. This is a recurring theme you'll meet again in Session 5: real implementations must sample from exactly the right distribution, and subtle sampling mistakes (or side-channel leakage *during* sampling) can quietly undermine security guarantees that look airtight on paper.

4.4 A Fully Worked Numeric Example

Example: Search-LWE by Hand, $n = 3$, $q = 17$

Let the (unknown, to an attacker) secret be $s = (2, 9, 1) \in \mathbb{Z}_{17}^3$, and let the error distribution χ produce values in $\{-1, 0, 1\}$ only (a toy stand-in for a discrete Gaussian). Suppose four samples are generated:

a_i	$\langle a_i, s \rangle \bmod 17$	e_i	$b_i = \langle a_i, s \rangle + e_i \bmod 17$
$(1, 0, 0)$	2	$+0$	2
$(0, 1, 0)$	9	-1	8
$(1, 1, 1)$	12	$+1$	13
$(3, 2, 1)$	$6 + 18 + 1 = 25 \equiv 8$	0	8

An attacker only ever sees the a_i (left column) and b_i (right column) — *not* the middle two columns. Notice that sample 1 alone, $b_1 = 2$, looks like it might reveal $s_1 = 2$ directly — and here it does, since e_1 happened to be 0. But sample 2 gives $b_2 = 8$, while the true $s_2 = 9$: the noise $e_2 = -1$ has already shifted the visible answer away from the truth by one full step, with no way for an outside observer to know whether the "off-by-one" is coming from s_2 or from e_2. Multiply that ambiguity across hundreds of dimensions and thousands of samples, and brute-force guessing collapses — this is the essence of why LWE is hard even though each individual equation looks almost solvable.

Remark: The Exact Same Example, Written as One Matrix Equation

Stack the four a_i rows from the table above into a matrix, exactly as the defbox's "row by row" remark describes — this is the $A \in \mathbb{Z}_{17}^{4\times 3}$ of the formal definition, with $m = 4$ samples and secret dimension $n = 3$:

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \\ 3 & 2 & 1 \end{pmatrix}, \qquad s = \begin{pmatrix} 2 \\ 9 \\ 1 \end{pmatrix}, \qquad e = \begin{pmatrix} 0 \\ -1 \\ 1 \\ 0 \end{pmatrix}, \qquad b = As + e \bmod 17 = \begin{pmatrix} 2 \\ 8 \\ 13 \\ 8 \end{pmatrix}.$$

Multiplying out As row by row reproduces exactly the middle column of the table above $(2, 9, 12, 8 \bmod 17)$, and adding e then reduces mod 17 reproduces the last column $(2, 8, 13, 8)$ — the two views really are the same computation. An attacker in the search-LWE game is handed only A and b (the matrix and the right-hand column) and must recover s; in the decision-LWE game, they're handed A and *either* this b *or* four independent uniform values in $\mathbb{Z}_{17}$, and must say which.

Exercise: Try This Before Next Lecture

Using the same A rows as above but a *different* secret $s' = (2, 8, 1)$ (differs from s only in the middle coordinate), recompute $b_1, \ldots, b_4$ with the same errors e_i. Compare the resulting b_i values to the table above. How many of the four samples end up identical to the original table, purely by coincidence of the noise? What does this tell you about how many samples an attacker needs before guessing becomes hopeless?

Remark: Making "Brute Force Collapses" Concrete

In the toy example above, each error e_i took one of only 3 values, and there were 4 samples. A brute-force attacker who doesn't know the errors could, in principle, try every combination of errors, undo them, and check whether the resulting system solves consistently. The number of combinations to try is $(\text{number of error values})^{(\text{number of samples})}$. The table below shows how fast this explodes as the problem scales toward realistic sizes (real schemes use hundreds of samples, and a Gaussian χ with far more than 3 realistic values — this table deliberately keeps 3 values fixed just to isolate the effect of *sample count* growing):

Samples	Combinations (3^{samples})	Roughly
4 (our example)	81	trivial by hand
20	≈ 3.5 billion	a laptop, seconds
100	$\approx 5 \times 10^{47}$	far beyond any computer on Earth
500 (realistic scale)	astronomically larger still	meaningless to even state

This is only a crude illustration (real attacks are smarter than raw brute force — that's what lattice-reduction algorithms like LLL and BKZ are for), but it captures the right intuition: **the ambiguity introduced by unknown noise compounds multiplicatively with every additional equation**, which is exactly the opposite of ordinary linear algebra, where more equations make life *easier*, not harder.

Example: Decision-LWE: Can You Spot the Real Samples?

Here are two sets of (a_i, b_i) pairs over $\mathbb{Z}_{17}$. One set was generated as genuine LWE samples $b_i = \langle a_i, s \rangle + e_i$ for some fixed secret s; the other set has every b_i replaced by an independent uniformly random value in $\mathbb{Z}_{17}$, with no relationship to a_i at all.

Set X		Set Y	
a_i	b_i	a_i	b_i
$(1, 0, 0)$	2	$(1, 0, 0)$	11
$(0, 1, 0)$	8	$(0, 1, 0)$	3
$(1, 1, 1)$	13	$(1, 1, 1)$	6

Try to decide, just by looking, which set is "real" LWE and which is uniform noise. (Set X is, in fact, the real one — it's the same data as our search-LWE example above.) **The point of this exercise is that you can't tell, and neither can a computer, efficiently** — both sets look like an unremarkable list of numbers in $\mathbb{Z}_{17}$. That indistinguishability, formalized, *is* decision-LWE. It's also exactly the property real encryption schemes lean on: a ciphertext under LWE-based encryption should be computationally indistinguishable from random noise to anyone without the secret key.

4.5 Why LWE Is Believed Hard: the Lattice Connection

Remark: LWE Is Bounded-Distance Decoding (BDD), a Cousin of CVP

Define the lattice $\Lambda_q(A) = \{\, y \in \mathbb{Z}_q^m : y \equiv As \pmod{q} \text{ for some } s \in \mathbb{Z}_q^n \,\}$ (all vectors reachable as A times *some* secret, viewed as a lattice via the ring structure of $\mathbb{Z}_q$ from Lecture 1). The vector $b = As + e$ is, by construction, a point $As \in \Lambda_q(A)$ that has been nudged by a *small* vector e. Recovering s from b is therefore exactly **Bounded-Distance Decoding**: given a target b that is guaranteed to be *unusually close* to some lattice point (closer than a typical random point would be), find that lattice point. This is CVP's easier, promise-bearing cousin — and, exactly as with CVP in Section 2.2, solving it reliably requires something like a good basis for $\Lambda_q(A)$. The whole point of choosing A uniformly at random is that no such good basis is visible to an attacker; only whoever *generated* s (or knows a matching trapdoor) has an edge.

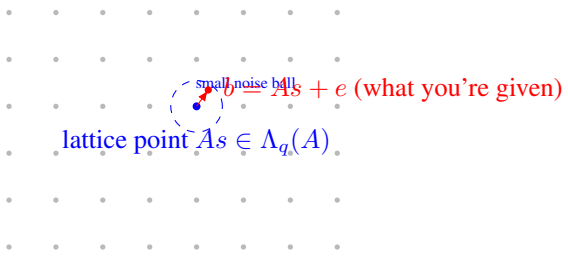


Figure 3: LWE as Bounded-Distance Decoding: b is guaranteed to land inside a small ball around some lattice point As — unlike general CVP (Figure 1), where the target could be anywhere. That extra promise ("close, not just closest") is what makes BDD tractable *with a good basis* and still believed hard *without one*.

Fact: Regev's Worst-Case-to-Average-Case Reduction (Statement Only)

A landmark 2005 result (Oded Regev) shows: if there existed an efficient algorithm that solves *average-case, randomly-generated* decision-LWE instances, that algorithm could be turned into an efficient algorithm for *worst-case* GapSVP and other hard lattice problems (via a quantum reduction; later work gave classical reductions for related problems). In plain terms: **LWE's hardness is not a bet made up from nothing** — it inherits its credibility from the same worst-case lattice hardness Lecture 1 built toward. This is exactly why cryptographers trust random LWE instances, generated fresh for every key, to be hard: hardness on average is tied by proof to hardness in the worst case. We take this as given; the proof itself is well beyond this course.

Looking Ahead: Search vs. Decision — Also Provably Equivalent

It's also a known theorem (not proven here) that search-LWE and decision-LWE are polynomially equivalent: an efficient solver for one can be converted into an efficient solver for the other. This matters practically because security *proofs* for encryption schemes are usually stated against decision-LWE ("ciphertexts are indistinguishable from random"), while *attacks* usually aim at search-LWE ("recover the actual secret key"). Session 5 will use both framings, so it's worth knowing they're two views of the same underlying hardness.

5 From Plain LWE to Ring-LWE

5.1 The Problem With Plain LWE: Size

A single LWE public key with security parameter n requires an $m \times n$ matrix A over $\mathbb{Z}_q$ — roughly n^2 numbers. For cryptographically secure parameters (n in the many hundreds), that's an enormous public key, and every encryption/decryption operation costs a full matrix-vector multiplication, $O(n^2)$ work. Real deployed schemes need public keys measured in kilobytes and operations fast enough for a TLS handshake or a smartcard, not this.

Example: Putting Real Numbers On "Too Big"

Take a security-relevant dimension, $n = 1024$, with q a few thousand (so each number needs about 12 bits ≈ 1.5 bytes to store). A plain-LWE public key needs roughly $n^2 = 1{,}048{,}576$ such numbers — over 1.5 megabytes, just for one public key, before even discussing ciphertexts. Compare that to a real ML-KEM-1024 public key, which is about $1{,}568$ *bytes* in total — roughly a thousand times smaller. That gap is exactly what Ring-LWE and Module-LWE close, by replacing an $n \times n$ grid of independent numbers with a much smaller number of *structured* ring elements that each silently represent n numbers at once (Section 5.3's remark makes precise how).

5.2 The Fix: Move Into a Polynomial Ring

Recall from Lecture 1's "Ring" section and its preview box: Kyber and Dilithium do arithmetic inside $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ — polynomials of degree $< n$ with coefficients in $\mathbb{Z}_q$, where powers of x wrap around using the rule $x^n \equiv -1$.

Definition: The Ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$

Elements of R_q are polynomials $a(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{n-1} x^{n-1}$ with each coefficient $a_i \in \mathbb{Z}_q$. Addition is ordinary coefficient-wise addition mod q. Multiplication is ordinary polynomial multiplication, *followed by reduction*: whenever a term x^n appears, replace it with -1 (and more generally $x^{n+k} \to -x^k$), then reduce every coefficient mod q. This makes R_q a finite ring with exactly q^n elements — and, crucially, a single element of R_q packs n numbers into one object that a computer can multiply in roughly $O(n \log n)$ time using the Number-Theoretic Transform (NTT, a finite-field cousin of the FFT) — we will not need NTT details in this course, just the fact that it's the reason Ring-LWE is fast in practice.

Example: Multiplying Two Small Polynomials in R_q, $n = 4$, $q = 17$

Let $n = 4$, so we reduce using $x^4 \equiv -1$, and work mod $q = 17$. Take $a(x) = 1 + 2x$ and $c(x) = 3 + x^3$ in R_{17}. Ordinary polynomial multiplication first:

$$a(x)\, c(x) = (1 + 2x)(3 + x^3) = 3 + x^3 + 6x + 2x^4.$$

Now reduce using $x^4 \equiv -1$: the term $2x^4$ becomes $2 \cdot (-1) = -2$. So we're left with

$$3 + 6x + x^3 - 2 \;=\; 1 + 6x + 0x^2 + x^3.$$

Finally reduce coefficients mod 17 (here every coefficient is already in $\{0, \ldots, 16\}$, so nothing changes): the product is $1 + 6x + x^3 \in R_{17}$. Notice the "wraparound" step ($x^4 \to -1$) is exactly what folds the high-degree term back down to degree < 4 — this is the polynomial-ring analogue of "carrying" in ordinary mod-q arithmetic.

5.3 Ring-LWE, Formally

Definition: Ring Learning With Errors (RLWE)

Fix $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ and an error distribution χ over R_q (coefficients drawn independently from a small distribution). Let $s \in R_q$ be a fixed secret ring element. A search-RLWE sample is a pair $(a, b) \in R_q \times R_q$ where a is uniform random and

$$b = a \cdot s + e \pmod{q}, \qquad e \leftarrow \chi.$$

Search-RLWE asks for s given many such pairs; decision-RLWE asks to distinguish (a, b) from a uniformly random pair in $R_q \times R_q$.

Remark: One Ring-LWE Sample $\approx n$ Plain-LWE Samples

Compare the two definitions side by side: plain LWE needs an entire matrix $A \in \mathbb{Z}_q^{m \times n}$ to state m equations. A *single* Ring-LWE pair (a, b), because a and s are each secretly a length-n coefficient vector multiplied together via the structured polynomial product above, is algebraically equivalent to about n ordinary LWE-style equations bundled into one ring multiplication. This is exactly where the size and speed win comes from — and exactly why the ring's extra structure is a trade-off: it buys efficiency at the cost of a stronger (though still, so far, unbroken) hardness assumption than plain LWE, since the extra algebraic structure is, in principle, more surface area for a future attack to exploit.

6 Module-LWE: the Middle Ground Kyber and Dilithium Actually Use

Definition: Module Learning With Errors (MLWE)

Fix R_q as above and a small integer k (the **module rank**). The secret is now a length-k vector of ring elements, $\vec{s} \in R_q^k$. A sample is $(\vec{a}, b)$ with $\vec{a} \in R_q^k$ uniform random and

$$b = \langle \vec{a}, \vec{s} \rangle + e \pmod{q} = \sum_{i=1}^{k} a_i s_i + e, \qquad e \leftarrow \chi.$$

Remark: MLWE Sits Exactly Between Plain LWE and Ring-LWE

Set $k = 1$ and MLWE collapses to Ring-LWE exactly. Let $n = 1$ (no ring structure at all — just plain integers) and MLWE collapses toward plain LWE. Choosing $k > 1$ with a modest per-ring dimension n gives designers a dial: more of the speed benefit of rings, without committing to the single largest, most algebraically rigid ring dimension. This is precisely why Kyber and Dilithium use MLWE with a moderate fixed $n = 256$ and vary k (Kyber uses $k \in \{2, 3, 4\}$ for its three security levels) instead of varying n directly — a design choice we will see in full in Session 3.

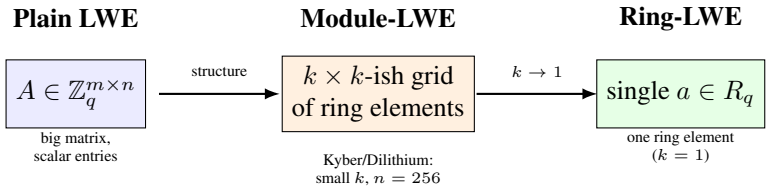


Figure 4: Module-LWE is a size/speed dial between the two extremes: no ring structure at all (plain LWE, left) and maximal ring structure (Ring-LWE, right). Kyber and Dilithium sit in the middle deliberately.

7 Bridge to Session 3: A Real Public Key *Is* an (M)LWE Sample

Looking Ahead: What You'll Recognize Next Session

When we open up Kyber (ML-KEM) in Session 3, its key generation will look almost exactly like the MLWE definition above: pick a random public matrix (of ring elements) $\mathbf{A}$, pick a small secret $\vec{s}$ and small error $\vec{e}$, and publish $\mathbf{t} = \mathbf{A}\vec{s} + \vec{e}$ as the public key — literally an MLWE sample, with $\vec{s}$ kept as the secret key. Encryption will use a *second*, independent MLWE-style sample to hide the message inside more noise. Once you can point at a real Kyber public key and say "that's $b = As + e$," the scheme stops looking like magic and starts looking like an application of exactly the object we defined today.

8 Summary Table

Problem	Given	Find / Decide
SVP	Basis B of lattice L	Shortest nonzero $v \in L$
CVP	Basis B, target $t \notin L$	Closest $v \in L$ to t
SIS	Random $A \in \mathbb{Z}_q^{n \times m}$	Short nonzero x: $Ax \equiv 0$
SIS hash $h_A(x) = Ax$	Same A, input $x \in \{0,1\}^m$	Compressed output in $\mathbb{Z}_q^n$; collisions $\Rightarrow$ SIS solutions
Search-LWE	$(A, b = As + e)$	Recover secret s
Decision-LWE	(A, b)	Is b "$As + e$" or uniform random?
Ring-LWE	$(a, b = as + e) \in R_q^2$	Recover $s \in R_q$
Module-LWE	$(\vec{a}, b) \in R_q^k \times R_q$	Recover $\vec{s} \in R_q^k$

9 Full List of Exercises (Assign as Homework Before Lecture 3)

Exercise: Homework Set

1. State, in your own words (2–3 sentences each), the difference between SVP and CVP, and the difference between the search and decision versions of a problem in general.
2. For $n = 1$, $m = 3$, $q = 13$, $A = \begin{pmatrix} 4 & 6 & 9 \end{pmatrix}$: find a nonzero, short $x \in \mathbb{Z}^3$ solving the SIS instance $Ax \equiv 0 \pmod{13}$ (Section 3 worked example, above). Confirm your answer by direct computation.
3. Complete the SIS-hash practice exercise in Section 3.1, if you have not already: find a genuine colliding pair for h_A and confirm it yields a valid SIS solution.
4. Redo the search-LWE worked example (Section 4.4) with secret $s = (2, 9, 1)$ but a *new* sample $a_5 = (2, 0, 1)$ and error $e_5 = 1$. Compute b_5. If you were only given a_5 and b_5 (not s or e_5), could you determine s from this single equation alone? Why not?
5. Explain in your own words why LWE would become *easy* (not hard) if the error e were removed entirely (i.e., $b = As$ exactly). Connect your answer to ordinary linear algebra.
6. In the ring $R_{17} = \mathbb{Z}_{17}[x]/(x^4 + 1)$: compute the product $(2 + x^2)(1 + 3x)$, showing the wraparound step explicitly (Section 5.2 worked example, above).
7. In 3–4 sentences: why does moving from plain LWE to Ring-LWE reduce key size roughly from $O(n^2)$ numbers to $O(n)$ numbers? Use the "one Ring-LWE sample $\approx n$ plain-LWE samples" remark (Section 5.3) in your explanation.
8. Module-LWE with $k = 1$ is exactly Ring-LWE, and (informally) R_q with $n = 1$ is exactly $\mathbb{Z}_q$. Using these two facts, explain why Module-LWE is fairly described as "a generalization that contains both plain LWE and Ring-LWE as special cases."
9. **Looking ahead:** Kyber-768 uses module rank $k = 3$ with per-ring dimension $n = 256$ and $q = 3329$. Without looking anything up yet, estimate how many total $\mathbb{Z}_q$-coefficients are packed into one Kyber-768 secret key $\vec{s} \in R_q^3$. (You'll check this answer in Session 3.)